02477 Bayesian Machine Learning 2026: Assignment 3

This is the last assignment in the Bayesian machine learning course 2026. The assignment is a group work of 2-4 students (please use the same groups as in assignment 1&2 if possible) and hand in via DTU Learn). The assignment is part of the curriculum, but handing in the assignment is **optional**. Handing in your solution is an opportunity to get feedback on your learning. The deadline for handing in is **3rd of May 23:59**. The solution must be handed in as a single **PDF** document.

Part 1: Regression modelling using mixture of experts

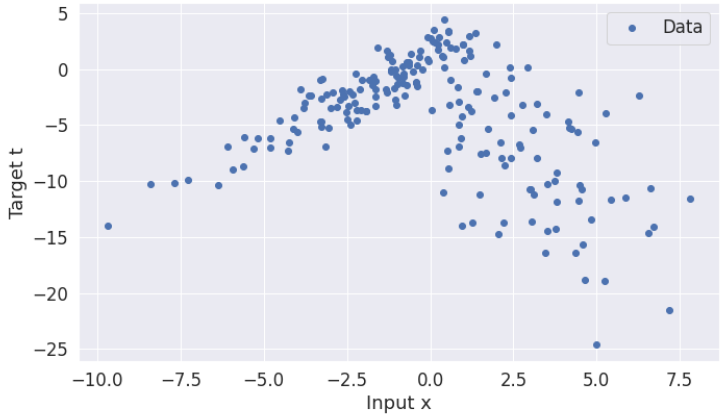

Figure 1: Dataset for regression

Consider the dataset for regression given in Figure 1. It is evident from the plot that there is a strong non-linear dependency between the target variables y_n and the input variables x_n. We also observe that the noise variance seems to depend on the input x. Consequently, assuming $y_n = y(x_n) + e_n$, where e_n is independent and identically distributed (i.i.d) noise would not be appropriate.

In this part, we will work with a so-called *Mixture of experts* (MoE) model for regression. The core idea is to model the dataset using several submodels, which are 'experts' in their own region of the input space. For example, the dataset in Figure 1 could be well-represented using two linear models: a linear model with positive slope and relative low noise variance for the 'left half' of the dataset and another linear model with negative slope and larger noise variance on the 'right half' of the dataset. Our goal is to simultaneously learn these two linear models as well as when to use which model.

To construct this model, we will consider two different linear models and then introduce a latent binary variable for each data point, $z_n \in \{0, 1\}$, to control which of the two linear models that should explain the data point y_n. That is, if $z_n = 0$ we assume y_n should be explained a linear model with parameters $\boldsymbol{w}_0$ and if $z_n = 1$, then y_n should be explained by a linear model with weights $\boldsymbol{w}_1$.

We model each z_n using Bernoulli distributions as follows

$$p(z_n|\pi_n) = \text{Ber}(z_n|\pi_n), \tag{1}$$

where $\pi_n = \sigma(\boldsymbol{v}^T \boldsymbol{x}_n)$ is the probability of $z_n = 1$, $\sigma(\cdot)$ is the logistic sigmoid function and $\boldsymbol{v}$ is a parameter vector to be estimated. That is, we basically model the latent z_n variable using a logistic regression model. We can set up a conditional likelihood as follows

$$p(y_n|x_n, z_n, \boldsymbol{w}_0, \boldsymbol{w}_1, \sigma_0^2, \sigma_1^2) = \begin{cases} \mathcal{N}(y_n|\boldsymbol{w}_1^T \boldsymbol{x}_n, \sigma_1^2) & \text{if} \quad z_n = 1, \\ \mathcal{N}(y_n|\boldsymbol{w}_0^T \boldsymbol{x}_n, \sigma_0^2) & \text{if} \quad z_n = 0, \end{cases} = \mathcal{N}(y_n|\boldsymbol{w}_{z_n}^T \boldsymbol{x}_n, \sigma_{z_n}^2) \tag{2}$$

where we also allow the noise variance to depend on z_n. We complete the model with generic priors

$$\tau, \sigma_0^2, \sigma_1^2 \sim \mathcal{N}_+(0,1) \tag{3}$$

$$\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v} \sim \mathcal{N}(\boldsymbol{0}, \tau^2 \boldsymbol{I}) \tag{4}$$

$$z_n | \boldsymbol{v} \sim \text{Ber}(\sigma(\boldsymbol{v}^T \boldsymbol{x}_n)) \tag{5}$$

$$y_n | z_n \sim \mathcal{N}(\boldsymbol{w}_{z_n}^T \boldsymbol{x}_n, \sigma_{z_n}^2), \tag{6}$$

where $\mathcal{N}_+(0,1)$ is the half-normal distribution. This leads to a joint model of the form

$$p(\boldsymbol{y}, \boldsymbol{w}_1, \boldsymbol{w}_0, \boldsymbol{v}, \boldsymbol{z}, \tau, \sigma_0, \sigma_1) = \left[\prod_{n=1}^N \mathcal{N}(y_n | \boldsymbol{w}_{z_n}^T \boldsymbol{x}_n, \sigma_{z_n}^2) \text{Ber}(z_n | \sigma(\boldsymbol{v}^T \boldsymbol{x}_n)) \right] \mathcal{N}(\boldsymbol{w}_0 | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \\ \mathcal{N}(\boldsymbol{w}_1 | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}(\boldsymbol{v} | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}_+(\tau^2 | 0,1) \mathcal{N}_+(\sigma_0 | 0,1) \mathcal{N}_+(\sigma_1 | 0,1). \tag{7}$$

The purpose is now to fit this model to the dataset in Figure 1 using MCMC. The dataset can be found on DTU Learn as loaded as follows:

```
data = jnp.load('./data_assignment3.npz')
x, y = data['x'], data['t']
```

To avoid handling the intercepts in the linear models explicitly, we will use the convention $\boldsymbol{x_n} = [x_n,\ 1]$.

Task 1.1: Marginalize out each z_n from to joint model in eq. (7) to obtain a joint distribution, where the likelihood for each observation is a mixture of two Gaussian distributions.

Hints: Use the sum rule to marginalize out z_n.

Solution

Denoting $p(\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}, \tau^2, \sigma_0, \sigma_1) = \mathcal{N}(\boldsymbol{w}_0 | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}(\boldsymbol{w}_1 | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}(\boldsymbol{v} | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}_+(\tau^2 | 0,1) \mathcal{N}_+(\sigma_0 | 0,1) \mathcal{N}_+(\sigma_1 | 0,1)$ and using the sum rule, we get

$$p(\boldsymbol{y}, \boldsymbol{w}_1, \boldsymbol{w}_0, \boldsymbol{v}, \boldsymbol{z}, \tau, \sigma_0, \sigma_1) \overset{(a)}{=} \sum_{\boldsymbol{z}} p(\boldsymbol{y}, \boldsymbol{w}_1, \boldsymbol{w}_0, \boldsymbol{v}, \boldsymbol{z}, \tau, \sigma_0, \sigma_1) \tag{8}$$

$$\overset{(b)}{=} \sum_{\boldsymbol{z}} \left[\prod_{n=1}^N \mathcal{N}(y_n | \boldsymbol{w}_{z_n}^T \boldsymbol{x}_n, \sigma_{z_n}^2) \text{Ber}(z_n | \sigma(\boldsymbol{v}^T \boldsymbol{x}_n)) \right] p(\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}, \tau^2, \sigma_0, \sigma_1) \tag{9}$$

$$\overset{(c)}{=} \left[\sum_{\boldsymbol{z}} \left[\prod_{n=1}^N \mathcal{N}(y_n | \boldsymbol{w}_{z_n}^T \boldsymbol{x}_n, \sigma_{z_n}^2) \text{Ber}(z_n | \sigma(\boldsymbol{v}^T \boldsymbol{x}_n)) \right] \right] p(\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}, \tau^2, \sigma_0, \sigma_1) \tag{10}$$

$$\overset{(d)}{=} \left[\prod_{n=1}^N \sum_{z_n} \mathcal{N}(y_n | \boldsymbol{w}_{z_n}^T \boldsymbol{x}_n, \sigma_{z_n}^2) \text{Ber}(z_n | \sigma(\boldsymbol{v}^T \boldsymbol{x}_n)) \text{d}z_n \right] p(\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}, \tau^2, \sigma_0, \sigma_1) \tag{11}$$

$$\overset{(e)}{=} \left[\prod_{n=1}^N (1 - \sigma(\boldsymbol{v}^T \boldsymbol{x}_n)) \mathcal{N}(y_n | \boldsymbol{w}_0^T \boldsymbol{x}_n, \sigma_0^2) + \sigma(\boldsymbol{v}^T \boldsymbol{x}_n) \mathcal{N}(y_n | \boldsymbol{w}_1^T \boldsymbol{x}_n, \sigma_1^2) \right] p(\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}, \tau^2, \sigma_0, \sigma_1), \tag{12}$$

where in (a) we use the sum rule, in (b) we substitute in the distributions concerning $\boldsymbol{z}$, in (c) we use the fact that $p(\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}, \tau^2, \sigma_0, \sigma_1)$ is independent of $\boldsymbol{z}$, in (d) we use the fact that $\mathbb{E}[xy] = \mathbb{E}[x]\mathbb{E}[y]$ when x, y are independent and finally, in (e) we compute the expectations with respect to each z_n. Therefore,

$$p(\boldsymbol{y}, \boldsymbol{w}_1, \boldsymbol{w}_0, \boldsymbol{v}, \boldsymbol{z}, \tau, \sigma_0, \sigma_1) \tag{13}$$

$$= \left[\prod_{n=1}^N (1 - \sigma(\boldsymbol{v}^T \boldsymbol{x}_n)) \mathcal{N}(y_n | \boldsymbol{w}_0^T \boldsymbol{x}_n, \sigma_0^2) + \sigma(\boldsymbol{v}^T \boldsymbol{x}_n) \mathcal{N}(y_n | \boldsymbol{w}_1^T \boldsymbol{x}_n, \sigma_1^2) \right] \tag{14}$$

$$\mathcal{N}(\boldsymbol{w}_0 | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}(\boldsymbol{w}_1 | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}(\boldsymbol{v} | \boldsymbol{0}, \tau^2 \boldsymbol{I}) \mathcal{N}_+(\tau^2 | 0,1) \mathcal{N}_+(\sigma_0 | 0,1) \mathcal{N}_+(\sigma_1 | 0,1) \tag{15}$$

Task 1.2: Implement a Python function to evaluate the marginalized log joint distribution

Hints: Let θ denote all the parameters of the model, i.e. $\theta = \{\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}, \tau, \sigma_0^2, \sigma_1^2\}$, then implement a function that takes θ and returns $\ln p(\boldsymbol{y}, \boldsymbol{w}_1, \boldsymbol{w}_0, \boldsymbol{v}, \tau, \sigma_0, \sigma_1)$.

Solution There are many ways to implement the log joint, e.g.

```
from scipy.stats import norm

sigmoid = lambda x: 1./(1 + jnp.exp(-x))

def halfnormal(x):
    if x < 0:
        return 0
    else:
        return 2*norm.pdf(x, 0, 1)

def log_halfnormal(x):
    if x < 0:
        return -jnp.inf
    else:
        return jnp.log(2) + norm.logpdf(x, 0, 1)

def unpack(params):
    w1 = params[:2]
    w2 = params[2:4]
    v = params[4:6]
    tau = params[6]
    sigma1 = params[7]
    sigma2 = params[8]
    return w1, w2, v, tau, sigma1, sigma2

def log_likelihood(X, y, params):
    w1, w2, v, tau, sigma1, sigma2 = unpack(params)

    g = X@v
    f1 = X@w1
    f2 = X@w2

    pi = sigmoid(g)

    if tau > 0 and sigma1 > 0 and sigma2 > 0:
        # could be implemented numerically more robust via logsumexp
        return jnp.log((1-pi)*norm.pdf(y, f1, sigma1) + pi*norm.pdf(y, f2, sigma2)).sum()
    else:
        return -jnp.inf

def log_prior(params):
    w1, w2, v, tau, sigma1, sigma2 = unpack(params)
    log_p = log_halfnormal(tau) + log_halfnormal(sigma2) + log_halfnormal(sigma1)
    log_p += norm.logpdf(w1, 0, tau).sum() + norm.logpdf(w2, 0, tau).sum() + norm.logpdf(v, 0,

    return log_p
```

```
# precompute design matrix
N = len(x)
X = jnp.column_stack((x, jnp.ones(N)))

# define log joint
log_joint_data = lambda X, y, params: log_likelihood(X, y, params)  + log_prior(params)
log_joint = lambda p: log_joint_data(X, y, p)
```

Task 1.3: Run a Metropolis-Hasting sampler to infer all parameters. Explain the settings you used (number of iterations, proposal distribution etc).

Hints: Feel free to use the Metropolis-Hasting code from the exercises. Plot the trace for all parameters to assess convergence (convergence diagnostics might be tricky due to the multimodality of the model). Compute the posterior mean of the weights $\boldsymbol{w}_0, \boldsymbol{w}_1, \boldsymbol{v}$, and plot the resulting function on top of the data as a sanity check. Make sure to initialize the sampler using a valid parameter vector, i.e. positive scale parameters $\sigma_0, \sigma_1, \tau > 0$.

Solution

```
num_params = 9
tau = 0.1
num_iter = 50000
seed = 123
params = jnp.ones(num_params)
samples, accept = metropolis(log_joint, num_params, tau, num_iter, theta_init=params, seed=123)
```

Figure 2 plots the trace of all 9 parameters of the model obtained using 50000 iterations with a Gaussian proposal distribution with std. dev. 10^{-1}.

Task 1.4: Report the posterior mean and 95% credibility intervals for all parameters.

Solution

We can estimate means and intervals as follows

```
params_samples_after_warmup = samples[25000:, :]
for i in range(num_params):
    # compute mean
    m = jnp.mean(params_samples_after_warmup[:, i])
    # intervals
    lower, upper = jnp.percentile(params_samples_after_warmup[:, i], jnp.array([2.5, 97.5]))
    print(f'{param_names[i]:15s} Mean = {m:+4.3f}\tInterval = [{lower:+4.3f}, {upper:+4.3f}]')
```

which yields

```
w_{0,0}         Mean = +1.471     Interval = [+1.342, +1.601]
w_{0,1}         Mean = +0.984     Interval = [+0.613, +1.369]
w_{1,0}         Mean = -1.735     Interval = [-2.289, -1.251]
w_{1,1}         Mean = -2.502     Interval = [-4.518, -0.536]
v_{0}           Mean = +2.379     Interval = [+1.590, +3.138]
v_{1}           Mean = -1.547     Interval = [-2.735, -0.725]
\tau            Mean = +1.747     Interval = [+1.003, +2.719]
\sigma0         Mean = +1.399     Interval = [+1.237, +1.589]
\sigma1         Mean = +4.500     Interval = [+3.904, +5.150]
```

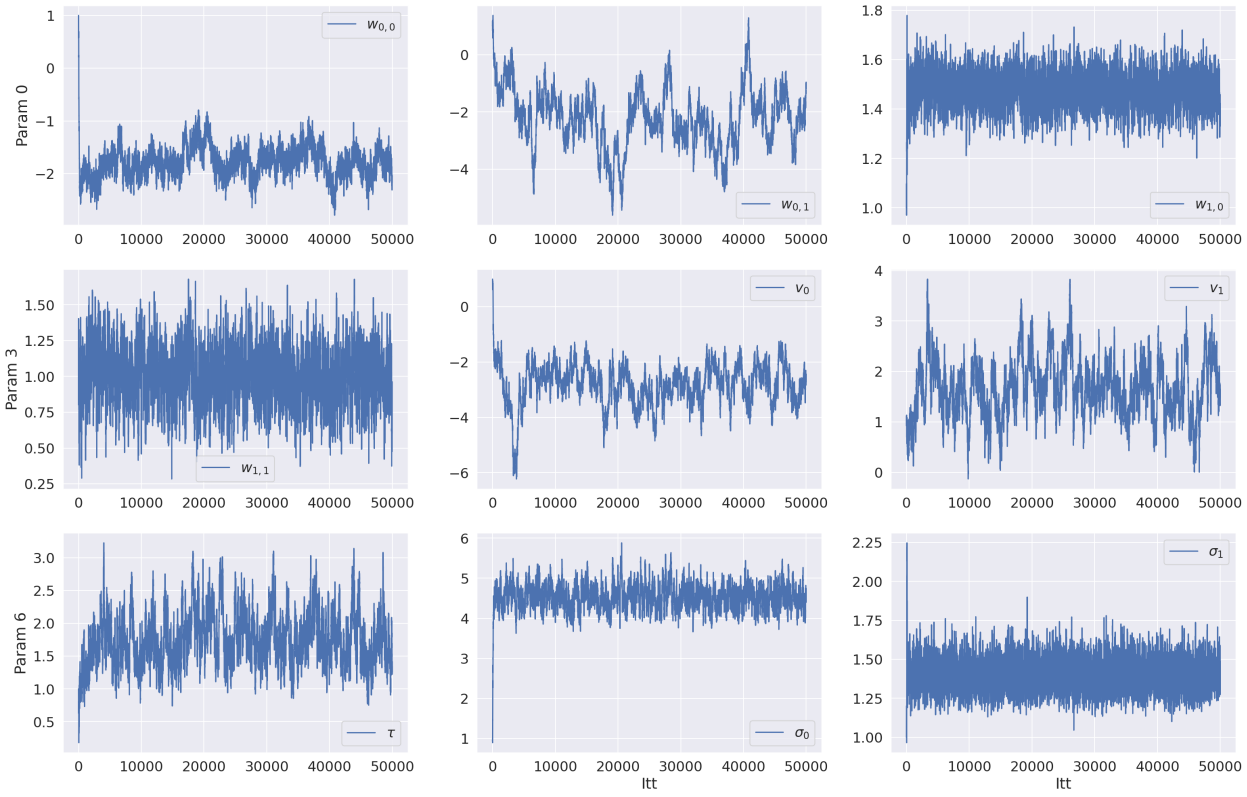

Figure 2: Trace of MCMC run for all 9 parameters.

Task 1.5: For $x \in [-12, 12]$, plot posterior predictive distribution for $p(\pi^*|\boldsymbol{y}, \boldsymbol{x}^*)$, $p(y^*|\boldsymbol{y}, \boldsymbol{x}^*, z^* = 0)$ and $p(y^*|\boldsymbol{y}, x, z^* = 1)$ on top of the data.

Hints: For inspiration on how to plot these distributions using samples, see the exercise on Gibbs sampling for change point detection.

Solution

Given the set of posterior samples, we can first compute the posterior samples of $\pi^*_{(i)}$, $y^*|z^* = 0$, $y^*|z^* = 1$, where (i) indicated the i'th sample:

$$\pi^*_{(i)} = \sigma(\boldsymbol{v}^T_{(i)}\boldsymbol{x}^*) \tag{16}$$

$$y^*_{(i)}|z^* = 0 \sim \mathcal{N}\left(\left(\boldsymbol{w}_0^{(i)}\right)^T \boldsymbol{x}^*, \left(\sigma_0^{(i)}\right)^2\right) \tag{17}$$

$$y^*_{(i)}|z^* = 1 \sim \mathcal{N}\left(\left(\boldsymbol{w}_1^{(i)}\right)^T \boldsymbol{x}^*, \left(\sigma_1^{(i)}\right)^2\right) \tag{18}$$

$$\tag{19}$$

and then compute the means and intervals using the `numpy.percentile`-function.

```
# extract samples for plotting p(pi^*|y, x^*) using pi^* = sigma(v^T x^*)
w0_samples = params_samples_after_warmup[:, 0:2].T
w1_samples = params_samples_after_warmup[:, 2:4].T
v_samples = params_samples_after_warmup[:, 4:6].T
sigma0_samples = params_samples_after_warmup[:, 7]
sigma1_samples = params_samples_after_warmup[:, 8]

# prep inputs
xp = jnp.linspace(-12, 12, 1000)
Xp = jnp.column_stack((xp, jnp.ones(len(xp))))

# prep keys
key = random.PRNGKey(123)
key1, key2 = random.split(key)

# compute p(pi^*|y, x^*)
pi_star = sigmoid(Xp@v_samples)
pi_star_lower, pi_star_upper = jnp.percentile(pi_star, jnp.array([2.5, 97.5]), axis=1)
pi_star_mean = jnp.mean(pi_star, axis=1)

# compute p(y^*|y, x^*, z^*=0)
y_star_z0 = Xp@w0_samples + sigma0_samples*random.normal(key1, shape=sigma0_samples.shape)
y_star_z0_lower, y_star_z0_upper = jnp.percentile(y_star_z0, jnp.array([2.5, 97.5]), axis=1)
y_star_z0_mean = jnp.mean(y_star_z0, axis=1)

# compute p(y^*|y, x^*, z^*=1)
y_star_z1 = Xp@w1_samples + sigma1_samples*random.normal(key2, shape=sigma1_samples.shape)
y_star_z1_lower, y_star_z1_upper = jnp.percentile(y_star_z1, jnp.array([2.5, 97.5]), axis=1)
y_star_z1_mean = jnp.mean(y_star_z1, axis=1)

# plot
fig, ax = plt.subplots(1, 2, figsize=(25, 10))
```

```
ax[0].plot(xp, pi_star_mean, 'b-', label='$p(\\pi^*|\\mathbf{y}, x^*)$')
ax[0].fill_between(xp, pi_star_lower, pi_star_upper, color='b', alpha=0.3)
ax[1].plot(xp, y_star_z0_mean, 'g-', label='$p(y^*|\\mathbf{y}, x^*, z^*=0)$')
ax[1].fill_between(xp, y_star_z0_lower, y_star_z0_upper , color='g', alpha=0.3)
ax[1].plot(xp, y_star_z1_mean, 'r-', label='$p(y^*|\\mathbf{y}, x^*, z^*=1)$')
ax[1].fill_between(xp, y_star_z1_lower, y_star_z1_upper , color='r', alpha=0.3)

for i in range(2):
    ax[i].legend()
    ax[i].set(xlabel='x')
ax[0].plot(x, 0.7 + 0.05*y, 'o', alpha=0.5, label='Shifted and scale data')
ax[1].plot(x, y, 'o', alpha=0.5, label='data')
ax[0].legend()
ax[1].legend()
```

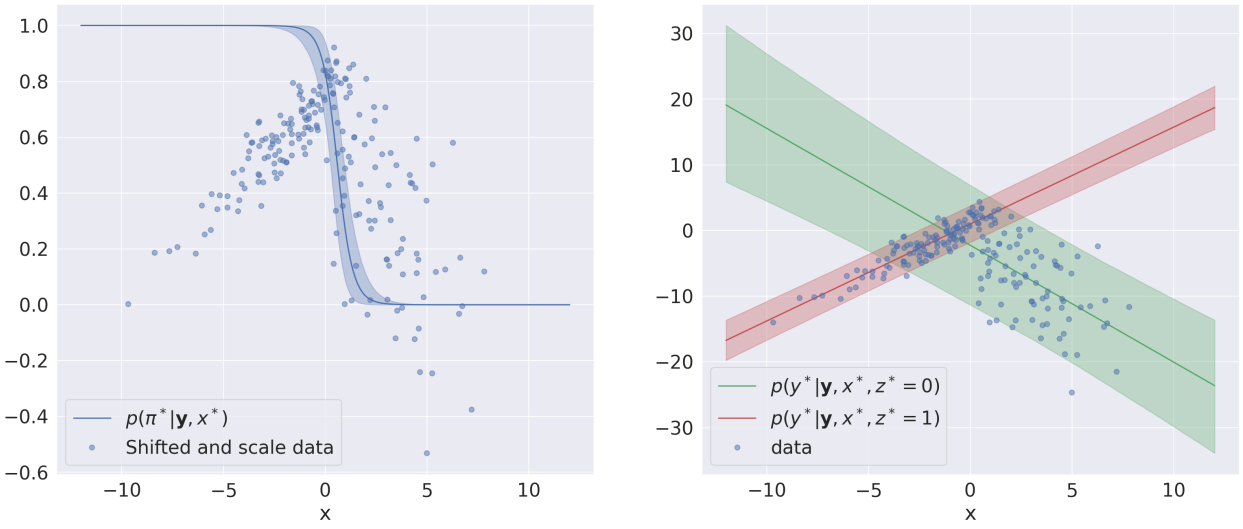

Figure 3: Plots for task 2.5

From the plot in Figure 3, we can see that the "change point" of the data roughly matches the position where $\pi^* = 0.5$ as expected (note that the data has been shifted and scale in the left panel to make this more clear). We can also that two conditional linear models appear to fit the left and right most part of the data nicely.

Task 1.6: For $x \in [-12, 12]$, plot posterior predictive distribution for $p(y^*|\boldsymbol{y}, \boldsymbol{x}^*)$

Solution

To compute the posterior predictive distribution, we repeat the same procedure as above, except that we now sample z^* as well.

```
# sample y^* based on posterior samples
def predictive_sample(key, params):
    # split key
    key_z, key_y1, key_y2 = random.split(key, num=3)
    w1_, w2_, v_, tau_, sigma1_, sigma2_ = unpack(params)

    pi = sigmoid(Xp@v_)
    z = random.binomial(key_z, 1, pi)
```

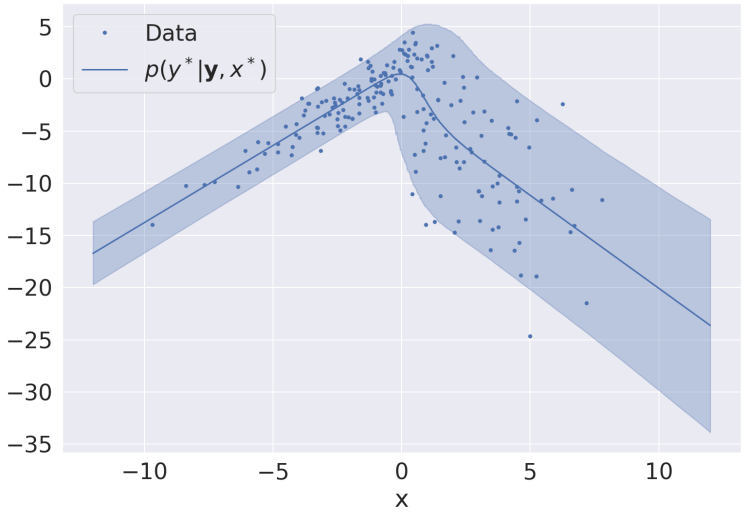

Figure 4: Plot for task 2.6

```
    y1 = Xp@w1_ + sigma1_*random.normal(key_y1)
    y2 = Xp@w2_ + sigma2_*random.normal(key_y2)
    return (1-z)*y1 + z*y2

# collect statistics
key = random.PRNGKey(123)
keys = random.split(key, num=len(params_samples_after_warmup))
y_samples = jnp.vstack([predictive_sample(key, p) for key, p in zip(keys, params_samples_after_warmu
y_mean = y_samples.mean(0)
y_lower, y_upper = jnp.percentile(y_samples, jnp.array([2.5, 97.5]), axis=0)
# plot
fig, ax = plt.subplots(1, 1, figsize=(12, 8))
ax.plot(x, y, '.', label='Data')
ax.plot(xp, y_mean, 'b-', label='$p(y^*|\\mathbf{y}, x^*)$')
ax.fill_between(xp, y_lower, y_upper, alpha=0.3, color='b')
ax.set(xlabel='x')
ax.legend()
```